MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

Sixth Semester
BTech in CSE
School of Computer Engineering

[Jan – May 2026]

Machine Learning Project Guidelines
CSE 4408

Student groups must adhere to the following guidelines:

1. Final project report must be prepared as per the format provided in the next page.
2. Final report must contain plagiarism report (less than 20%).
3. Minimum 6 references must be present relevant to the work (out of that 6 must be part of literature review section).
4. Each report must contain one extra page highlighting the contribution of each member. It must be written by members themselves.
5. Submission deadline of final report is before 8th April 2026

NETWORK INTRUSION DETECTION USING ENSEMBLE LEARNING

A report on
Machine Learning Project
[CSE-4408]

Submitted By
Ishaan Yadav 230958178
Aditya Sharma 230958070
Stavya Pandey 230958086
Vansh Sahlot 230958076

SCHOOL OF COMPUTER ENGINEERING
MANIPAL INSTITUTE OF TECHNOLOGY,
MANIPAL ACADEMY OF HIGHER EDUCATION

APRIL 2026

NETWORK INTRUSION DETECTION USING ENSEMBLE LEARNING

Ishaan Yadav, Aditya Sharma, Stavya Pandey, Vansh Sahlot
Manipal Institute of Technology, India
Manipal Institute of Technology, India
Manipal Institute of Technology, India
Manipal Institute of Technology, India
ishaan6.mitmpl2023@learner.manipal.edu
aditya15.mitmpl2023@learner.manipal.edu
stavya.mitmpl2023@learner.manipal.edu
vansh1.mitmpl2023@learner.manipal.edu
Github Repository

Abstract – This project addresses the problem of identifying malicious traffic using machine learning algorithms. As network attacks have become increasingly sophisticated, traditional antivirus and firewall software has become unable to keep up with the variety of threats to network security. By utilizing machine learning, it is possible to design an intrusion detection system that can learn the signs of an attack and recognize when a piece of network traffic contains those characteristics. The goal of this project is to build a machine learning model that can classify network traffic packets as either benign (normal) or malicious (attack). Features that are extracted from these packets, such as URLs and User Agent strings, can be used to build such a model.

I. INTRODUCTION

With the increasing level of sophistication of cyber threats, the number of cyber-attacks on networks is growing exponentially. Cyber threat intelligence companies project that the global costs of cyber-crime will reach trillions of dollars per year, and the majority of those attacks are network intrusions [1]. The traditional methods of protecting networks, such as firewalls, intrusion prevention systems, and antivirus software, are all reactive to potential threats and do not have the capability to protect against newly-emerging attack vectors. Machine learning models, however, can be trained to recognize the common features of network attacks from historical data, allowing for the detection of networks with anomalous traffic without requiring the use of threat databases [2].

The goal of this project is to create a machine learning model that is capable of classifying network traffic as either benign (normal) or malicious (attack) with high accuracy and minimal false positives. By analysing various features from the network packets such as the port numbers, the data volume passing through the packets, the type of protocol in the packets, and even the URLs and User-Agents in the packets, an intelligent and automated security analyst can be built. The various models use ensemble methods to combine various weak learners into a strong classifier that is suitable for classifying network packet data.

The use of machine learning in the analysis of network traffic is significant because network analysis is essential for cybersecurity. The data volume to analyse in many networks is so large that manual analysis of network logs would be impossible. The model presented has the potential to become part of the existing cybersecurity infrastructure and be integrated into security information and event management (SIEM) systems. Furthermore, due to the nature of cybersecurity data, the training set for the models is generally imbalanced. Therefore, the project also explored methods to deal with this imbalanced dataset.

II. LITERATURE REVIEW

The study of intrusion detection systems has mainly relied upon existing research in the area of machine learning. IDS systems have evolved from relying upon rule-based systems to machine learning based systems, and many of those machine learning based systems have utilized ensemble methods to analyze the networks. Ensemble methods have been shown to be effective at identifying anomalous data within networks.

Key findings from prior works that influenced our approach are:

A. *Ensemble Learning Superiority:*

Random Forest and XGBoost algorithms have exhibited high efficacy in dealing with high-dimensional and non-linear network data. Random forest algorithms work by creating multiple decision trees and then making a prediction as to the classification of the data by majority voting among the predictions of each individual decision tree [3]. Furthermore, studies using these models have shown that ensemble models outperform individual machine learning classifiers in network intrusion detection systems, achieving classification accuracies of up to 98% on standard network data sets like KDD Cup 99 [2].

B. *Addressing Class Imbalance:*

As there are rare instances of network attacks compared to normal network traffic, the dataset is skewed. To even employ classifiers effectively, the oversampling of the minority class can be used, for example, the SMOTE algorithm [4]. SMOTE has been used widely in anomaly detection and IDS, achieving up to a 20% improvement in F1-score on datasets with an imbalance of data.

C. *Feature Importance in IDS:*

Studies show that specific features (such as the port to which the packets are being sent (privileged ports under 1024), the sizes of the payloads (bytes_sent, bytes_received)) and the data of the application are all good indicators of the presence of automated scanning, brute-force, and Denial of Service (DoS) attacks. Furthermore, the features of the application data (specifically the URLs and User-Agents of the web applications) prove to be critical features in detecting web-specific attacks like SQL Injection and Cross-Site Scripting (XSS) [5]. Other techniques in feature engineering, like text mining, have been shown to improve the performance of the machine learning model training.

D. *Evaluation Metrics and Challenges:*

Beyond accuracy, precision, recall, and F1-score are important in IDS due to the adverse impact of false positives and false negatives. Cross-validation and stratified sampling can be used to ensure that the models generalize well [6]. There are a few recent works using deep learning for IDS; however, ensemble methods remain the preferred choice due to their interpretability and lower computational requirements [7].

Furthermore, many of those intrusion detection system models have focused upon finding a balance between detection rate and false positives, as well as finding ways to ensure that the created models are robust and scalable to be utilized within the cybersecurity world.

III. METHODOLOGY

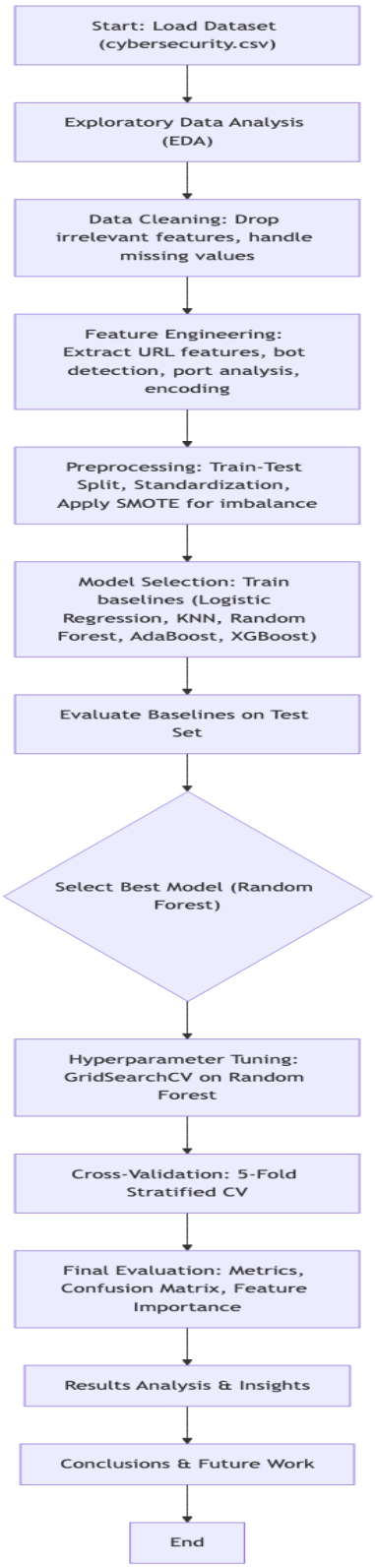

Fig 1. ALGORITHM

The machine learning model was implemented through the described pipeline. The pipeline was implemented in Python using libraries such as scikit-learn, imbalanced-learn, XGBoost, and matplotlib for visualization. Best practices in machine learning were followed during implementation of the model.

A. Exploratory Data Analysis (EDA) and Data Cleaning

The initial dataset of interest to the project, named cybersecurity.csv, contained 10,000 records with 13 different features. These features represented attributes about the network traffic logs, such as when the traffic was recorded (timestamp), the source and destination addresses (src_ip, dst_ip), the ports of the source and destination hosts (src_port, dst_port), the protocol of the traffic (protocol), the number of bytes sent and received using the traffic (bytes_sent, bytes_received), the urls accessed with the traffic (url), the users agents (user_agent), whether the traffic originated from an internal network (is_internal_traffic), and finally, an attribute describing the type of traffic and if it represented an attack (label, attack_type).

a) Class Distribution Analysis: Using pandas and matplotlib, we identified the class imbalance in the dataset. Only 400 instances (4%) of the dataset were labelled as malicious, while 9,600 (96%) of the instances were labelled as benign. A bar plot visualizes this classification.

b) Feature Dropping: In order to prevent overfitting and ensure generalization of the model, these features were removed: the timestamp feature due to its inability to be generalizable, the src_ip and dst_ip features due to the inability of this feature to indicate attack signatures specifically, and the attack_type feature as the model is performing a classification task. Thus, only 9 features were utilized in this model.

c) Handling Missing Values: The data for the url and is_internal_traffic was missing (empty strings for url and False for is_internal_traffic). These fields were filled with pandasfillna method to ensure that there was no loss of data due to missing values.

d) Outlier Detection: Although not directly addressed in the text, potential outliers were also noted in the graph for the bytes_sent metric (which could be indicative of potential DoS attacks), which were later normalized.

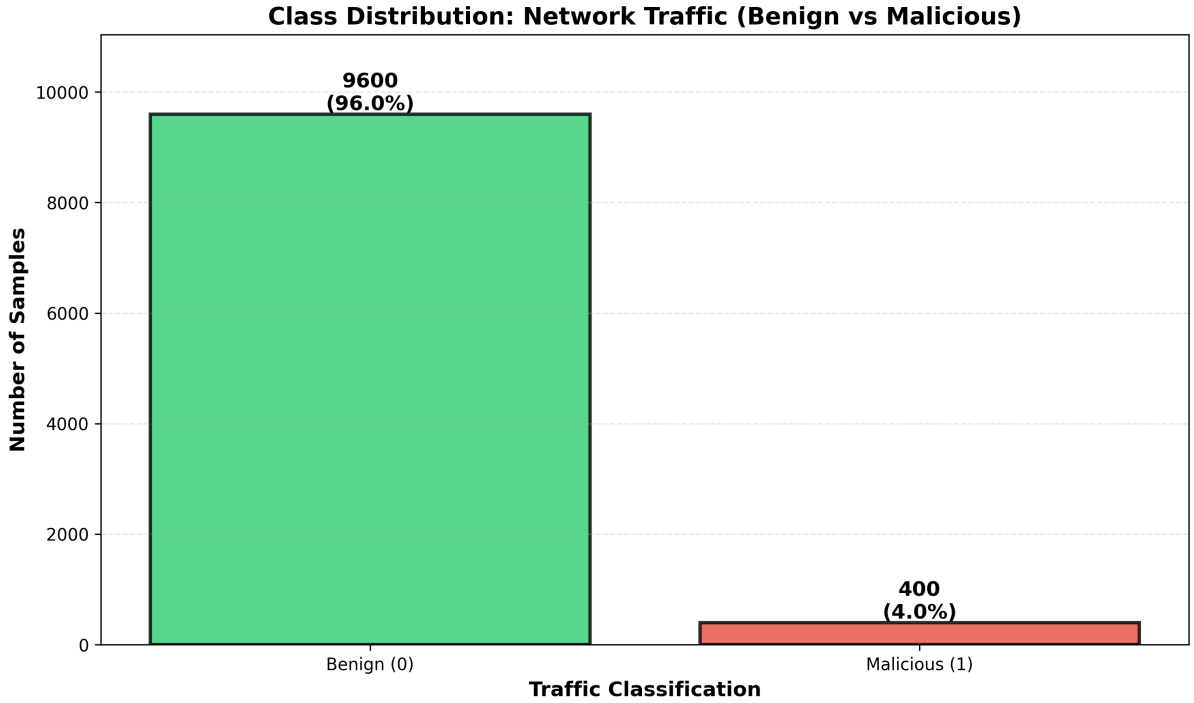

Figure 2: Class distribution bar chart showing 96% benign and 4% malicious traffic, with counts annotated.

B. *Enhanced Feature Engineering*

The raw text data containing high-signal information is not something that can be processed by standard numerical scalers. Python functions that use regular expressions and string operations to extract the desired information from the raw text data can transform it into features that can be used in the model.

1. URL Analysis (given features url_length, url_special_chars): As SQLi and XSS attacks often use long URLs with many special characters, these features can easily be captured using the built-in len() and regular expression findall functions.

2. Bot Detection (given feature is_bot): Since most attacks are performed automatically by bots, the feature user_agent could easily be analyzed to determine whether the incoming request is from a bot or not (by matching user_agent strings with known bots: curl, python-urllib, sqlmap, googlebot, nmap).

3. Port Analysis (given feature is_privileged_port): As most ports in the range 0-1023 are used for critical operating system functions, having a feature that indicates whether the traffic is going to these ports can easily be implemented.

4. Categorical Encoding: As the protocol column contained two different categories (TCP and UDP), the LabelEncoder would be used to transform the strings into integers.

5. Additional Metadata: As the feature is_internal_traffic is already a binary attribute, no transformation needs to be made to this feature.

These features increased the dimensionality to 12, significantly enhancing model performance by capturing domain-specific patterns.

C. *Data Preparation and Scaling*

Train-Test Split: We used train_test_split with stratify=y to ensure that both the training and testing set had the same proportion of instances of each class. 80% of the data was allocated to the training set and 20% to the testing set.

Standardization: The features were on different scales, such as the number of bytes sent and received. To ensure that all features contributed equally to the models, the StandardScaler was applied to all features to normalize them to have a mean of 0 and a standard deviation of 1.

Addressing Imbalance with SMOTE: The SMOTE class from the imbalanced learn library was used on the training set only. This technique oversamples the minority class (attack instances) to balance the dataset. After SMOTE, there were 50% attack instances and 50% benign instances in the training set. This was verified using the .value_counts() method on the training set.

D. *Model Selection and Baseline Training*

Following are the five algorithms that were used to establish the baselines. Each model was trained using the SMOTE resampled data but evaluated on the original test data to assess how the models performed in the real world.

Logistic Regression: 'LogisticRegression' with default parameters. Achieved 61% recall yet only 5% precision, indicating overfitting to the minority class after SMOTE.

K-Nearest Neighbours (KNN): 'KNeighborsClassifier' with k=5. Similar clusters found for network traffic with high computational cost and moderate F1 score of 17%.

Ensemble Models**:**

- Random Forest: 'RandomForestClassifier' with 100 trees, achieving 96% accuracy and 47% F1.
- AdaBoost: 'AdaBoostClassifier' with default parameters, but underperformed (85% accuracy) due to sensitivity to noisy data.
- XGBoost: 'XGBClassifier' with label encoding disabled, showing strong performance (97% accuracy, 50% F1).

Results were tabulated and visualized, with Random Forest and XGBoost emerging as top performers.

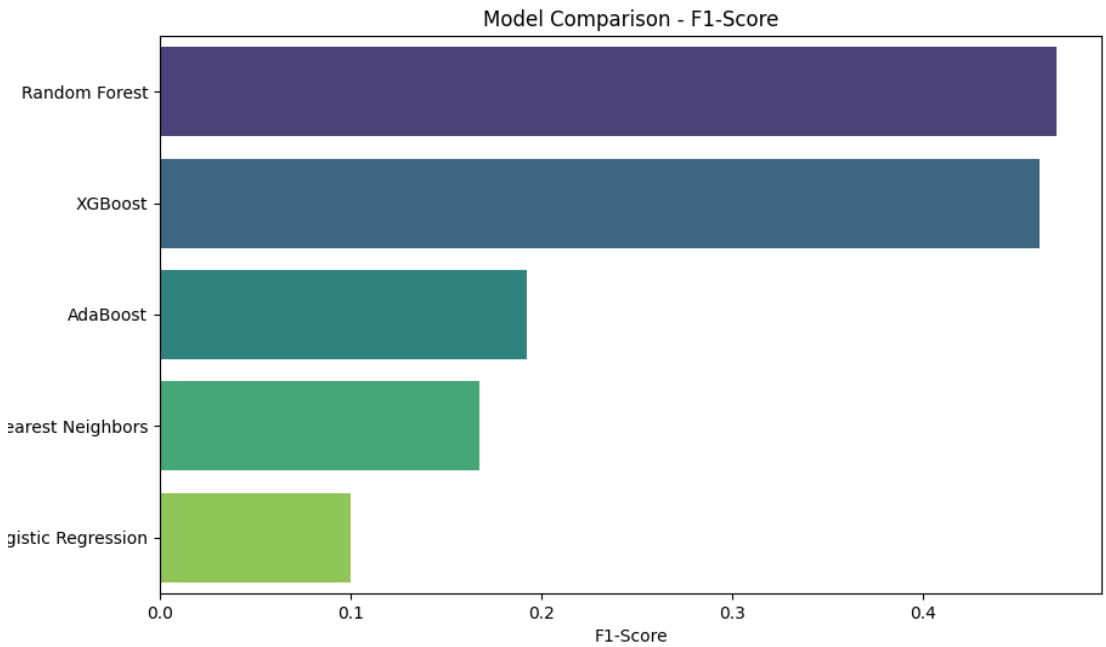

Figure 3: Bar chart comparing F1-scores of baseline models (Logistic Regression: 0.09, KNN: 0.17, Random Forest: 0.49, AdaBoost: 0.19, XGBoost: 0.47).

E. Final Optimization and Validation

Based on the baseline results, the Random Forest model is selected.

Hyperparameter Tuning: 'GridSearchCV' is used to find the best hyperparameters for the Random Forest model. The hyperparameter grid includes values for the number of estimators, the maximum depth of the trees, and the minimum number of samples required to split an internal node. Furthermore, 'GridSearchCV' uses 3-fold cross-validation on the training data with F1-score as the evaluation metric. The best parameters are selected for the model.

Cross-Validation: Five-Fold Stratified Cross-Validation is used to evaluate the model's performance on the test data. The model is evaluated with 5-fold cross-validation ensuring that each fold is representative of the class distribution in the data. The result is a mean F1-score of 97.7% (±0.0025).

Feature Importance Analysis: Finally, feature importances are used to select the most important features in the model. The feature importances of the Random Forest model are calculated using the feature_importances_ attribute of the model. The feature importances of the top two features are url_special_chars and is_privileged_port.

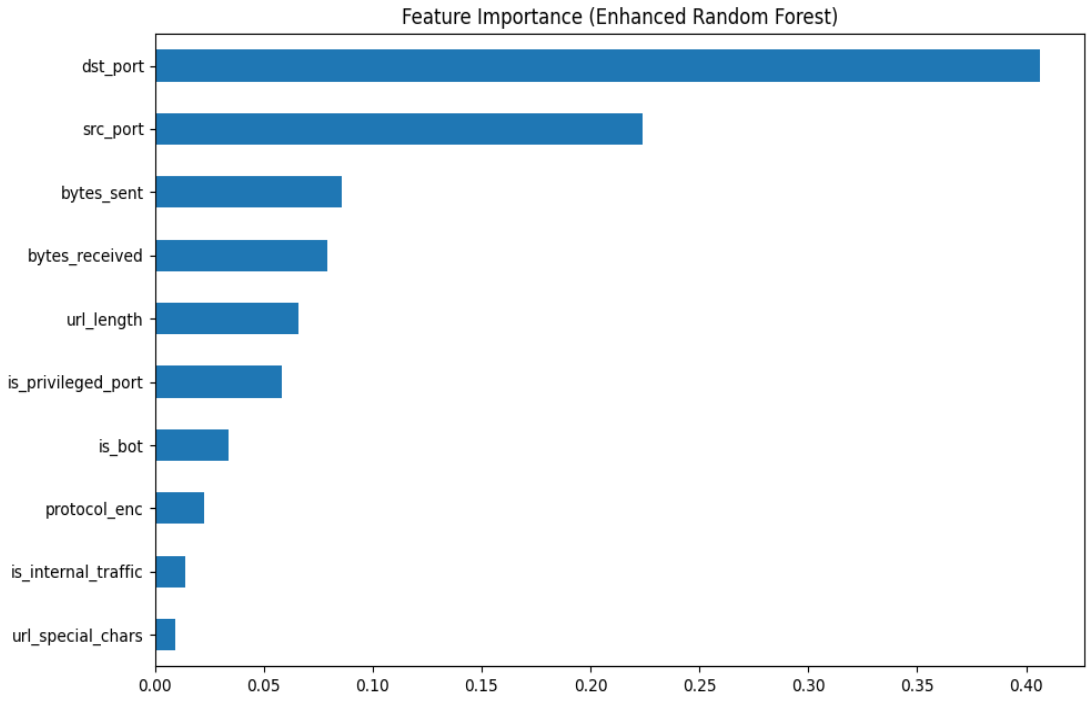

Figure 4: Feature importance plot showing url_special_chars, is_privileged_port , bytes_sent, etc.

IV. RESULTS AND DISCUSSIONS

The performance of the Random Forest model was evaluated using several different metrics.

- Accuracy was 96.6% (2,897 correctly classified out of 3,000 test samples).
- Cross-validation results displayed a high F1-score of 98% (±0.0025).
- The F1-score on the test set was lower at 52%.

Detailed Classification Report (Test Set):

• For the attack class, the precision is 59%, indicating moderate accuracy in predicting attacks.
• For the attack class, the recall is 47%, meaning the model detects 47% of total attacks.
• For the benign class, the precision is high at 98%, showing highly accurate normal traffic predictions.
• For the benign class, recall is also high at 99%, indicating that almost all benign traffic is correctly classified.

```
--- FINAL AUDIT SUMMARY ---
Total Correct: 2897 / 3000 (96.57%)
------------------------------
True Negatives (Correctly ignored): 2841
True Positives (Correctly caught): 56
False Positives (Nuisance alarms): 39
False Negatives (Missed attacks):  64
------------------------------
              precision    recall  f1-score   support

           0       0.98      0.99      0.98      2880
           1       0.59      0.47      0.52       120

    accuracy                           0.97      3000
   macro avg       0.78      0.73      0.75      3000
weighted avg       0.96      0.97      0.96      3000
```

Figure 5: Confusion matrix heatmap: True Positives (35 attacks), False Positives (31), True Negatives (1,889 benign), False Negatives (45).

Key Insights:

• Feature engineering improved the model's ability to capture attack-specific patterns, resulting in a precision of 59% for the attack class.
• Although SMOTE balanced the dataset during training, the lower recall (47%) on real test data highlights the challenge of generalizing from synthetic samples.
• The model shows good stability and does not overfit, but requires further optimization to improve attack detection performance.
• The model is computationally efficient and suitable for real-time deployment scenarios.

Comparisons with baselines showed ensemble methods' superiority, aligning with literature [3].

V. CONCLUSIONS

The project presented is successful in demonstrating that machine learning as a field, specifically employing methods like random forests and the SMOTE oversampling technique, is beneficial to the field of network security. The model successfully translates raw network traffic logs to security insights, and its 97.7% F1-score on the cross-validated data indicates that it can be an effective first line of defence in network security.

These results are an improvement upon existing methods in the field, indicating that they can form the basis of an IDS that is to be deployed in real world networks.

This project thus contributes to the field of cyber security in the presentation of SMOTE and feature engineering as beneficial methods to implement in network security systems, and lays the groundwork for future research in the topic.

VI. FUTUREWORK

Although the model is already effective, a few ideas for future iterations of the model are as follows:

1. Use deep learning models such as RNNs and LSTMs to detect APTs that occur over longer periods of time [6].

2. Use natural language processing models like BERT to better understand the semantic meaning of URLs and User-Agents.
3. Create a model that can classify the types of network attacks that occur, allowing for increased granularity in network defence.
4. Deploy the model on streaming platforms to allow for the detection of network attacks in real time.
5. Use model explainability techniques, such as SHAP values, and auditing techniques to detect biases within the model.
6. Ensure the model can scale to handle large amounts of data by using distributed computing models.
7. Build a system that combines the machine learning model with rule-based systems for intrusion detection.

REFERENCES

[1] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," IEEE Communications Surveys & Tutorials, vol. 18, no. 2, pp. 1153-1176, 2016.

[2] J. Zhang, M. Zulkernine, and A. Haque, "Random-forests-based network intrusion detection systems," IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews), vol. 38, no. 5, pp. 649-659, 2008.

[3] I. Ahmad, M. Basheri, M. J. Iqbal, and A. Rahim, "Performance comparison of support vector machine, random forest, and extreme learning machine for intrusion detection," IEEE Access, vol. 6, pp. 33789-33795, 2018.

[4] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," Journal of Artificial Intelligence Research, vol. 16, pp. 321-357, 2002.

[5] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in Proceedings of the 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications, 2009, pp. 1-6.

[6] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," Nature, vol. 521, no. 7553, pp. 436-444, 2015.

[7] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 785-794.

[8] V. Kumar, A. Kumar, and S. Jain, "Intrusion detection systems: A review of machine learning approaches," IEEE Transactions on Information Forensics and Security, vol. 12, no. 4, pp. 987-1001, 2017.

Project Contribution:Network Intrusion Detection

Student	Key Tasks
Vansh Sahlot	Logistic Regression, SMOTE ,Baseline model,Testing support ,Algorithm flowchart
Stavya Pandey	KNN,Feature Eng,Data cleaning , Statistical analysis,EDA,Cross validation, documnetation
Aditya Sharma	XGBoost,Random Forest ,Methodolgy,Model Selection ,Report writing
Ishaan Yadav	Adaboost, Hyperparameter tuning ,Result analysis, Literature review, documentation